

Bases de données 2

Rouvrais Clément/ Maxime Brun/ Safiane Ras-el-Qdim/ Luc Martrenchar

ArtConnect

Livrable — Étape 2



1. Introduction

L'objectif de cette étape est de concevoir les modèles conceptuel et logique de la base de données ArtConnect, en partant des fonctionnalités identifiées à l'étape 1. La démarche suit les étapes classiques de la modélisation relationnelle :

- Identification des entités et attributs à partir du modèle Java
- Construction du MCD (Modèle Conceptuel de Données / ERD)
- Dérivation du MLD (Modèle Logique de Données) avec clés primaires, étrangères et contraintes
- Normalisation jusqu'en 3e Forme Normale (3FN)
- Écriture du script SQL de création de la base

2. Entités et attributs

Les entités suivantes ont été identifiées à partir des classes Java du modèle. Pour chaque entité, nous avons défini un identifiant technique (id AUTO_INCREMENT) afin d'éviter les problèmes liés aux clés naturelles (doublons, modifications de noms, etc.).

Entité	Attributs principaux	Remarques
artist	id, name, bio, birth_year, contact_email, phone, city, website, social_media, is_active	Clé naturelle candidate : name (UNIQUE). is_active permet de désactiver sans supprimer.
artwork	id, title, creation_year, type, medium, dimensions, description, price, status, artist_id	status est une enum (FOR_SALE, SOLD, EXHIBITED). artist_id FK vers artist.
discipline	id, name	Entité de référentiel. name est UNIQUE. Relation many-to-many avec artist et community_member.
gallery	id, name, address, owner_name, opening_hours, contact_phone, rating, website	rating est un DECIMAL(3,1). name est UNIQUE.
exhibition	id, title, start_date, end_date, description, gallery_id, curator_name, theme	gallery_id FK vers gallery. Contrainte : end_date >= start_date (vérifiée applicativement).
workshop	id, title, date_time, duration_minutes, max_participants, price, instructor_id, location, description, level	instructor_id FK vers artist. level : beginner / intermediate / advanced.
community_member	id, name, email, birth_year, phone, city, membership_type	email et name sont UNIQUE. membership_type : free ou premium.
booking	id, workshop_id, member_id, booking_date, payment_status	Contrainte UNIQUE (workshop_id, member_id) : un membre ne peut réserver qu'une fois par atelier.
review	id, member_id, artwork_id, rating, comment, review_date	rating : CHECK entre 1 et 5. UNIQUE (member_id, artwork_id).

En plus de ces 9 entités, 3 tables de jointure sont nécessaires pour les associations many-to-many :

- `artist_discipline` : relie `artist` et `discipline`
 - `member_discipline` : relie `community_member` et `discipline` (disciplines favorites)
- `exhibition_artwork` : relie `exhibition` et `artwork`

Voici les dépendances fonctionnelles de notre base de donnée :

artist

- `id` → `name`, `bio`, `birth_year`, `contact_email`, `phone`, `city`, `website`, `social_media`, `is_active`
- `name` → `id` (clé candidate UNIQUE)

artwork

- `id` → `title`, `creation_year`, `type`, `medium`, `dimensions`, `description`, `price`, `status`, `artist_id`
- `artist_id` → `artist` (FK)

discipline

- `id` → `name`
- `name` → `id` (UNIQUE)

gallery

- `id` → `name`, `address`, `owner_name`, `opening_hours`, `contact_phone`, `rating`, `website`
- `name` → `id` (UNIQUE)

exhibition

- `id` → `title`, `start_date`, `end_date`, `description`, `gallery_id`, `curator_name`, `theme`
- `gallery_id` → `gallery` (FK)
- (`start_date`, `end_date`) : contrainte `end_date` ≥ `start_date`

workshop

- `id` → `title`, `date_time`, `duration_minutes`, `max_participants`, `price`, `instructor_id`, `location`, `description`, `level`
- `instructor_id` → `artist` (FK)

community_member

- `id` → `name`, `email`, `birth_year`, `phone`, `city`, `membership_type`
- `email` → `id` (UNIQUE)
- `name` → `id` (UNIQUE)

booking

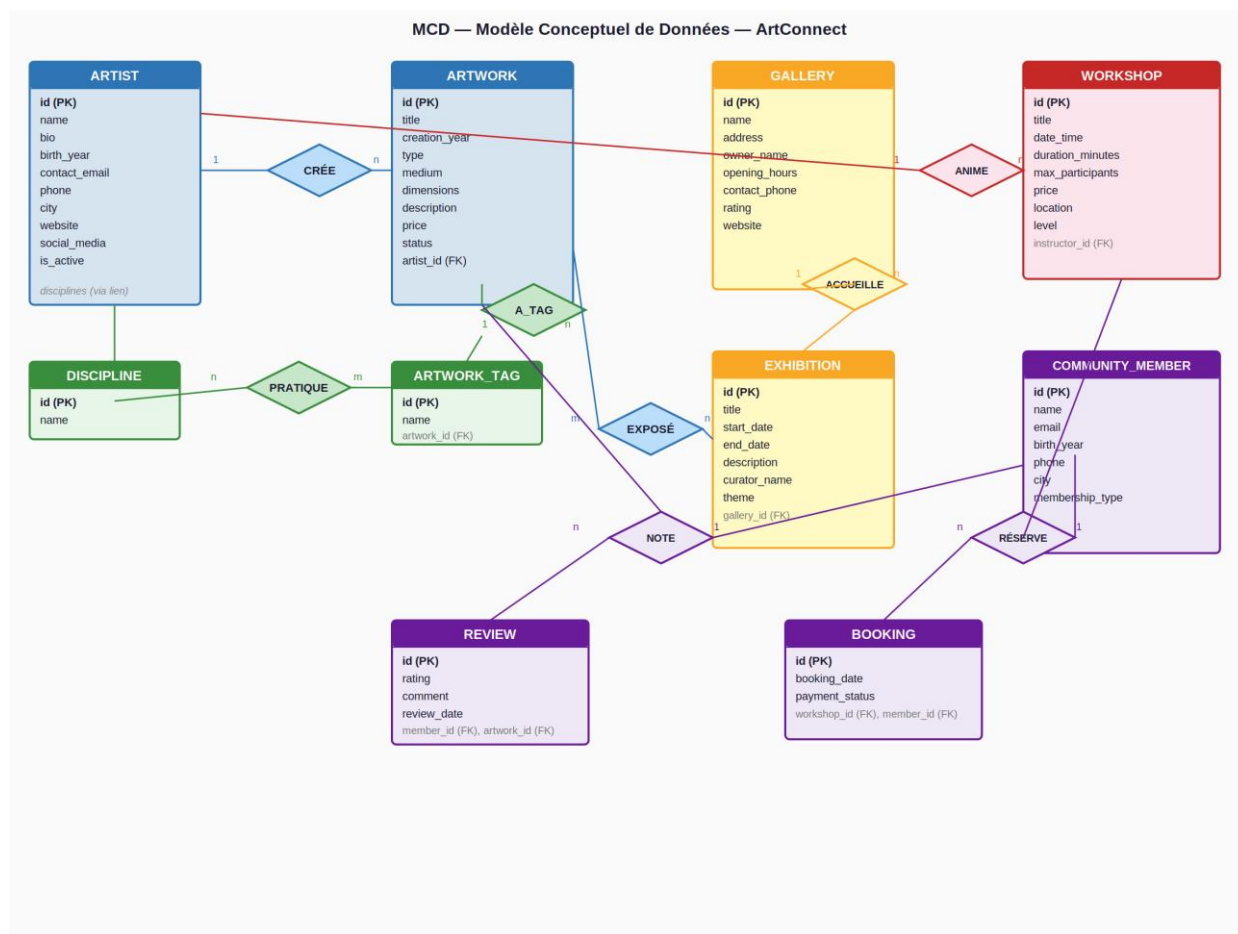
- `id` → `workshop_id`, `member_id`, `booking_date`, `payment_status`
- (`workshop_id`, `member_id`) → `id` (UNIQUE — un membre ne peut réserver qu'une fois par atelier)
- `workshop_id` → `workshop` (FK)
- `member_id` → `community_member` (FK)

review

- `id` → `member_id`, `artwork_id`, `rating`, `comment`, `review_date`
- (`member_id`, `artwork_id`) → `id` (UNIQUE)
- `member_id` → `community_member` (FK)
- `artwork_id` → `artwork` (FK)

3. Modèle Conceptuel de Données (MCD / ERD)

Le diagramme Entité-Relation ci-dessous représente les entités, leurs attributs et les associations entre elles. Les losanges représentent les associations avec leur cardinalité.



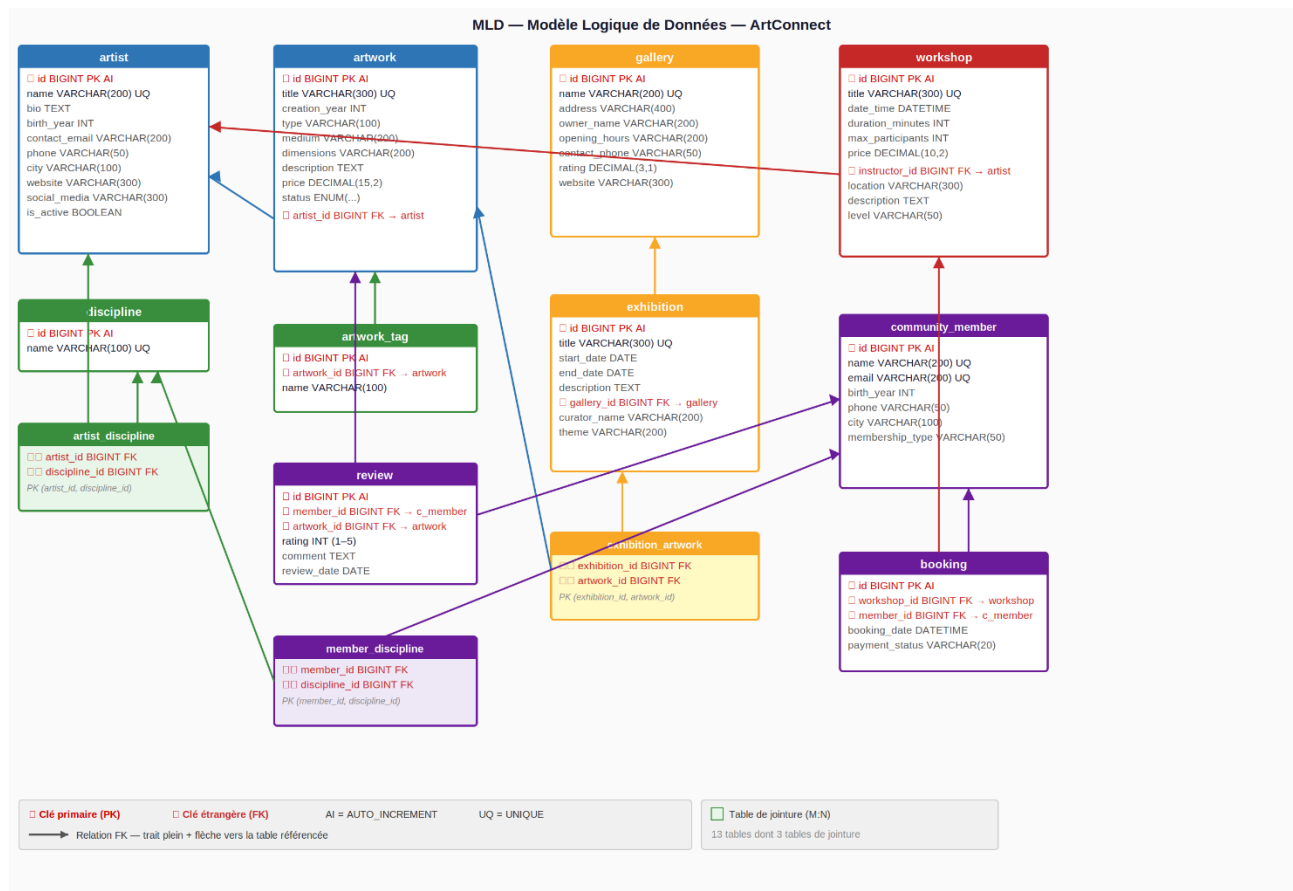
3.1 Description des associations

Association	Cardinalité	Description
CRÉE (artist → artwork)	1,N	Un artiste crée une ou plusieurs œuvres. Une œuvre est créée par exactement un artiste. Association binaire de type composition : la suppression d'un artiste entraîne la suppression de ses œuvres (ON DELETE CASCADE).
PRATIQUE (artist ↔ discipline)	M,N	Un artiste pratique une ou plusieurs disciplines artistiques. Une discipline peut être pratiquée par plusieurs artistes. Table de jointure artist_discipline.
A_TAG (artwork → artwork_tag)	1,N	Une œuvre peut avoir zéro ou plusieurs tags descriptifs. Un tag est attaché à une seule œuvre.
ACCUEILLE (gallery → exhibition)	1,N	Une galerie accueille zéro ou plusieurs expositions. Une exposition a lieu dans exactement une galerie.
EXPOSÉ (exhibition ↔ artwork)	M,N	Une exposition présente plusieurs œuvres. Une même œuvre peut figurer dans plusieurs expositions différentes. Table de jointure exhibition_artwork.

Association	Cardinalité	Description
ANIME (artist → workshop)	1,N	Un artiste anime zéro ou plusieurs ateliers (en tant qu'instructeur). Un atelier est animé par un seul artiste.
RÉSERVE (community_member + workshop → booking)	M,N	Un membre réserve zéro ou plusieurs ateliers. Un atelier peut avoir plusieurs réservations (dans la limite des places). Table booking avec attribut propre (booking_date, payment_status).
NOTE (community_member + artwork → review)	M,N	Un membre peut noter plusieurs œuvres. Une œuvre peut recevoir des notes de plusieurs membres. Contrainte d'unicité : un seul avis par (membre, œuvre).

4. Modèle Logique de Données (MLD)

Le MLD est obtenu par transformation systématique du MCD. Chaque entité devient une table, chaque association many-to-many devient une table de jointure, et les associations 1,N sont matérialisées par une clé étrangère dans la table du côté N.



4.1 Règles de transformation appliquées

- Toute entité → table avec id BIGINT AUTO_INCREMENT comme clé primaire
- Association 1,N → clé étrangère (FK) dans la table côté N
- Association M,N → table de jointure avec clé primaire composite (FK1, FK2)
- Attributs enum Java → type ENUM MySQL (status dans artwork)
- Dates Java (LocalDate) → DATE MySQL ; LocalDateTime → DATETIME
- Boolean Java → BOOLEAN MySQL (is_active dans artist)
- double Java → DECIMAL pour les montants financiers (price, rating)

4.2 Contraintes d'intégrité

Table	Type de contrainte	Détail
artist	UNIQUE	name — deux artistes ne peuvent avoir le même nom
artwork	UNIQUE	title — titre d'œuvre unique
artwork	FOREIGN KEY	artist_id → artist(id) ON DELETE CASCADE

Table	Type de contrainte	Détail
artwork	NOT NULL	price, status, artist_id
gallery	UNIQUE	name — nom de galerie unique
exhibition	UNIQUE	title — titre d'exposition unique
exhibition	FOREIGN KEY	gallery_id → gallery(id) ON DELETE CASCADE
workshop	UNIQUE	title — titre d'atelier unique
workshop	FOREIGN KEY	instructor_id → artist(id) ON DELETE CASCADE
community_member	UNIQUE	name, email — deux contraintes séparées
booking	UNIQUE composite	(workshop_id, member_id) — un seul booking par (atelier, membre)
booking	FOREIGN KEY ×2	workshop_id → workshop(id), member_id → community_member(id)
review	CHECK	rating BETWEEN 1 AND 5
review	UNIQUE composite	(member_id, artwork_id) — un seul avis par (membre, œuvre)
review	FOREIGN KEY ×2	member_id → community_member(id), artwork_id → artwork(id)

5. Normalisation jusqu'en 3e Forme Normale (3FN)

La normalisation vise à éliminer les redondances et anomalies d'insertion/mise à jour/suppression. Nous vérifions ici que chaque table du schéma respecte les trois premières formes normales.

5.1 Première Forme Normale (1FN)

Une table est en 1FN si tous ses attributs sont atomiques (non répétitifs, indivisibles).

Toutes les tables respectent la 1FN. Justifications :

- Les listes de disciplines (Artist.disciplines) et de tags (Artwork.tags) sont extraites vers des tables séparées (discipline, artwork_tag) — il n'y a donc pas de valeurs multivaluées dans une colonne.
- Les listes de bookings et reviews (CommunityMember.bookings, .reviews) sont représentées par des tables dédiées (booking, review).
- Chaque attribut contient une valeur atomique unique (ex. : contact_email contient un seul email, pas une liste).

5.2 Deuxième Forme Normale (2FN)

Une table est en 2FN si elle est en 1FN et que tout attribut non-clé dépend de la totalité de la clé primaire (pas de dépendances partielles). La 2FN ne concerne que les tables à clé composite.

Toutes les tables respectent la 2FN. Justifications :

- Tables à clé simple (id) : 2FN automatiquement satisfaite car il ne peut y avoir de dépendance partielle sur une clé à un seul attribut.
- Tables à clé composite : artist_discipline (artist_id, discipline_id) : aucun attribut non-clé → 2FN triviale.
- member_discipline (member_id, discipline_id) : idem, aucun attribut supplémentaire.
- exhibition_artwork (exhibition_id, artwork_id) : idem.
- booking : bien que workshop_id et member_id forment une clé candidate UNIQUE, la PK est id. Les attributs booking_date et payment_status dépendent du booking entier, pas seulement d'un des deux FK.

5.3 Troisième Forme Normale (3FN)

Une table est en 3FN si elle est en 2FN et qu'il n'existe pas de dépendances fonctionnelles transitives (un attribut non-clé dépendant d'un autre attribut non-clé).

Toutes les tables respectent la 3FN. Analyses table par table :

Table	Analyse des dépendances fonctionnelles	Verdict
artist	id → {name, bio, birth_year, contact_email, phone, city, website, social_media, is_active}. Aucune DF transitive : city ne détermine pas d'autre attribut dans la table.	3FN
artwork	id → {title, creation_year, type, medium, dimensions, description, price, status, artist_id}. artist_id est une FK, pas un attribut descriptif de l'œuvre. Aucune DF transitive.	3FN
discipline	id → name. Table à 2 attributs : 3FN triviale.	3FN

Table	Analyse des dépendances fonctionnelles	Verdict
gallery	id → {name, address, owner_name, opening_hours, contact_phone, rating, website}. Aucune DF transitive : address ne détermine pas owner_name.	3FN
exhibition	id → {title, start_date, end_date, description, gallery_id, curator_name, theme}. curator_name est un attribut propre de l'exposition (pas une FK vers une entité curator — volontairement simplifié).	3FN
workshop	id → {title, date_time, duration_minutes, max_participants, price, instructor_id, location, description, level}. Aucune DF transitive.	3FN
community_member	id → {name, email, birth_year, phone, city, membership_type}. Aucune DF transitive : membership_type ne dépend que de l'id, pas de city.	3FN
booking	id → {workshop_id, member_id, booking_date, payment_status}. payment_status dépend du booking, pas du workshop ou du membre seul.	3FN
review	id → {member_id, artwork_id, rating, comment, review_date}. rating et comment dépendent de la review, pas de artwork ou member seuls.	3FN

Choix de conception particulier : l'attribut curator_name dans exhibition stocke le nom du curateur sous forme de chaîne plutôt qu'une FK vers une table curator. Ce choix est justifié par la simplicité du périmètre fonctionnel actuel. Si la gestion des curateurs devenait une entité à part entière, une 4e entité serait créée et ce champ serait remplacé par curator_id FK.

6. Script SQL de création de la base

Le script suivant crée l'intégralité du schéma ArtConnect dans MySQL. Il est idempotent : les instructions CREATE DATABASE IF NOT EXISTS et CREATE TABLE IF NOT EXISTS permettent de le réexécuter sans erreur.

Pour exécuter le script : `mysql -u root -p < schema.sql`

```
-- =====
-- ArtConnectPro - Schéma MySQL
-- Exécuter en tant que : mysql -u root -p < schema.sql
-- =====

CREATE DATABASE IF NOT EXISTS artconnect_db
    CHARACTER SET utf8mb4
    COLLATE utf8mb4_unicode_ci;

USE artconnect_db;

-- -----
-- discipline
-- -----
CREATE TABLE IF NOT EXISTS discipline (
    id      BIGINT      NOT NULL AUTO_INCREMENT,
    name    VARCHAR(100) NOT NULL UNIQUE,
    PRIMARY KEY (id)
);

-- -----
-- artist
-- -----
CREATE TABLE IF NOT EXISTS artist (
    id          BIGINT      NOT NULL AUTO_INCREMENT,
    name        VARCHAR(200) NOT NULL UNIQUE,
    bio         TEXT,
    birth_year  INT,
    contact_email VARCHAR(200),
    phone       VARCHAR(50),
    city        VARCHAR(100),
    website     VARCHAR(300),
    social_media VARCHAR(300),
    is_active   BOOLEAN     NOT NULL DEFAULT TRUE,
    PRIMARY KEY (id)
);

-- -----
-- artist_discipline (many-to-many)
-- -----
CREATE TABLE IF NOT EXISTS artist_discipline (
    artist_id    BIGINT NOT NULL,
    discipline_id BIGINT NOT NULL,
    PRIMARY KEY (artist_id, discipline_id),
    FOREIGN KEY (artist_id) REFERENCES artist(id) ON DELETE CASCADE,
    FOREIGN KEY (discipline_id) REFERENCES discipline(id) ON DELETE CASCADE
);

-- -----
-- artwork
-- -----
CREATE TABLE IF NOT EXISTS artwork (
    id          BIGINT      NOT NULL AUTO_INCREMENT,
    title       VARCHAR(300) NOT NULL UNIQUE,
    creation_year INT,
    type        VARCHAR(100),
    medium      VARCHAR(200),
```

```

        dimensions    VARCHAR(200),
        description    TEXT,
        price          DECIMAL(15, 2) NOT NULL DEFAULT 0,
        status         ENUM('FOR_SALE', 'SOLD', 'EXHIBITED') NOT NULL DEFAULT 'FOR_SALE',
        artist_id      BIGINT          NOT NULL,
        PRIMARY KEY (id),
        FOREIGN KEY (artist_id) REFERENCES artist(id) ON DELETE CASCADE
    );

-- -----
-- artwork_tag
-- -----
CREATE TABLE IF NOT EXISTS artwork_tag (
    id          BIGINT          NOT NULL AUTO_INCREMENT,
    artwork_id BIGINT          NOT NULL,
    name        VARCHAR(100) NOT NULL,
    PRIMARY KEY (id),
    FOREIGN KEY (artwork_id) REFERENCES artwork(id) ON DELETE CASCADE
);

-- -----
-- gallery
-- -----
CREATE TABLE IF NOT EXISTS gallery (
    id          BIGINT          NOT NULL AUTO_INCREMENT,
    name        VARCHAR(200)   NOT NULL UNIQUE,
    address     VARCHAR(400),
    owner_name  VARCHAR(200),
    opening_hours VARCHAR(200),
    contact_phone VARCHAR(50),
    rating      DECIMAL(3, 1)  NOT NULL DEFAULT 0,
    website     VARCHAR(300),
    PRIMARY KEY (id)
);

-- -----
-- exhibition
-- -----
CREATE TABLE IF NOT EXISTS exhibition (
    id          BIGINT          NOT NULL AUTO_INCREMENT,
    title       VARCHAR(300) NOT NULL UNIQUE,
    start_date  DATE,
    end_date    DATE,
    description  TEXT,
    gallery_id  BIGINT          NOT NULL,
    curator_name VARCHAR(200),
    theme       VARCHAR(200),
    PRIMARY KEY (id),
    FOREIGN KEY (gallery_id) REFERENCES gallery(id) ON DELETE CASCADE
);

-- -----
-- exhibition_artwork (many-to-many)
-- -----
CREATE TABLE IF NOT EXISTS exhibition_artwork (
    exhibition_id BIGINT NOT NULL,
    artwork_id    BIGINT NOT NULL,
    PRIMARY KEY (exhibition_id, artwork_id),
    FOREIGN KEY (exhibition_id) REFERENCES exhibition(id) ON DELETE CASCADE,
    FOREIGN KEY (artwork_id)    REFERENCES artwork(id)    ON DELETE CASCADE
);

-- -----
-- workshop
-- -----
CREATE TABLE IF NOT EXISTS workshop (
    id          BIGINT          NOT NULL AUTO_INCREMENT,
    title       VARCHAR(300)   NOT NULL UNIQUE,
    date_time   DATETIME       NOT NULL,

```

```

    duration_minutes INT          NOT NULL DEFAULT 60,
    max_participants INT          NOT NULL DEFAULT 10,
    price            DECIMAL(10, 2) NOT NULL DEFAULT 0,
    instructor_id    BIGINT        NOT NULL,
    location         VARCHAR(300),
    description      TEXT,
    level            VARCHAR(50),
    PRIMARY KEY (id),
    FOREIGN KEY (instructor_id) REFERENCES artist(id) ON DELETE CASCADE
);

-- -----
-- community_member
-- -----
CREATE TABLE IF NOT EXISTS community_member (
    id            BIGINT          NOT NULL AUTO_INCREMENT,
    name          VARCHAR(200) NOT NULL UNIQUE,
    email         VARCHAR(200) NOT NULL UNIQUE,
    birth_year    INT,
    phone         VARCHAR(50),
    city          VARCHAR(100),
    membership_type VARCHAR(50) NOT NULL DEFAULT 'free',
    PRIMARY KEY (id)
);

-- -----
-- member_discipline (many-to-many : favorite disciplines)
-- -----
CREATE TABLE IF NOT EXISTS member_discipline (
    member_id      BIGINT NOT NULL,
    discipline_id  BIGINT NOT NULL,
    PRIMARY KEY (member_id, discipline_id),
    FOREIGN KEY (member_id) REFERENCES community_member(id) ON DELETE CASCADE,
    FOREIGN KEY (discipline_id) REFERENCES discipline(id) ON DELETE CASCADE
);

-- -----
-- booking
-- -----
CREATE TABLE IF NOT EXISTS booking (
    id            BIGINT          NOT NULL AUTO_INCREMENT,
    workshop_id   BIGINT          NOT NULL,
    member_id     BIGINT          NOT NULL,
    booking_date  DATETIME        NOT NULL DEFAULT CURRENT_TIMESTAMP,
    payment_status VARCHAR(20) NOT NULL DEFAULT 'PENDING',
    PRIMARY KEY (id),
    UNIQUE KEY uq_booking (workshop_id, member_id),
    FOREIGN KEY (workshop_id) REFERENCES workshop(id) ON DELETE CASCADE,
    FOREIGN KEY (member_id) REFERENCES community_member(id) ON DELETE CASCADE
);

-- -----
-- review
-- -----
CREATE TABLE IF NOT EXISTS review (
    id            BIGINT NOT NULL AUTO_INCREMENT,
    member_id     BIGINT NOT NULL,
    artwork_id    BIGINT NOT NULL,
    rating        INT      NOT NULL CHECK (rating BETWEEN 1 AND 5),
    comment       TEXT,
    review_date   DATE     NOT NULL DEFAULT (CURRENT_DATE),
    PRIMARY KEY (id),
    UNIQUE KEY uq_review (member_id, artwork_id),
    FOREIGN KEY (member_id) REFERENCES community_member(id) ON DELETE CASCADE,
    FOREIGN KEY (artwork_id) REFERENCES artwork(id) ON DELETE CASCADE
);

```

7. Conclusion

Le schéma ArtConnect comprend 13 tables (9 entités + 3 tables de jointure) couvrant l'ensemble des besoins fonctionnels identifiés à l'étape 1. Toutes les tables respectent la 3e Forme Normale, garantissant l'absence de redondances et la cohérence des données.

Les points forts du schéma :

- Clés primaires techniques (AUTO_INCREMENT) sur toutes les entités — robustesse face aux changements de données métier
- Contraintes UNIQUE sur les attributs naturels (name, email, title) — intégrité sans PK composite fragile
- ON DELETE CASCADE sur les FK enfants — cohérence automatique lors des suppressions
- Contrainte CHECK sur review.rating — validation à la couche base de données
- Contraintes UNIQUE composites sur booking et review — règles métier garanties au niveau SQL

Ce schéma sera la base de l'étape 3, lors de laquelle seront ajoutés les données d'exemple, les vues, les index, les triggers et les procédures stockées.